

```

%% Logistic Regression Code
%

% Below needs to be applied before testing.
%XTestBS = table2array(XTestBS);
%YTestBS = table2array(YTestBS);
%% Basic LR Model
% Fitlinear function being applied to training dataset.

rng(1);%
L1=fitlinear(XTrain1BS,YTrain1BS,"Learner","logistic",'KFold',10);

rng(1)
L1_loss = kfoldLoss(L1)
%% LR to find the optimum combination regularization method out of ridge and lasso
using bayesianoptimization.

%% Finds optimum value for lambda given we are using ridge regression, found by
% the equation.

%%https://www.mathworks.com/help/stats/bayesopt.html

rng(1);
FG = fitlinear(XTrain1BS,YTrain1BS,"Learner","logistic",'OptimizeHyperparameters',...
    {'Lambda','Regularization'},'HyperparameterOptimizationOptions', struct('Kfold',10,...
    'Optimizer','bayesopt','AcquisitionFunctionName','expected-improvement-plus',...
    'MaxObjectiveEvaluations', 30))
%% Try: Experimenting; to see what happens with 'expected improvement only'

HF = fitlinear(XTrain1BS,YTrain1BS,"Learner","logistic",'OptimizeHyperparameters',...
    {'Lambda','Regularization'},'HyperparameterOptimizationOptions', struct('Kfold',10,...
    'Optimizer','bayesopt','AcquisitionFunctionName','expected-improvement',...
    'MaxObjectiveEvaluations', 30))
%% LR Final model

rng(1);
L2 = fitlinear(XTrain1BS,YTrain1BS,"Learner","logistic","Lambda", 0.0045629,"Regularization",...
    "ridge",KFold=10);

L2_loss = kfoldLoss(L2)

%%
%% LR Final model- used in Test

% This code is the one used in the final model. The lambda value and the
% optimization type has been chosen by bayesian optimization.

rng(1);
L3 = fitlinear(XTrain1BS,YTrain1BS,"Learner","logistic","Lambda", 0.0045629,"Regularization",...
    "ridge")
%%
%
%% Performance Measures in LR

LR_Predict = predict(L3, XTestBS);
LR_Result = confusionmat(YTestBS,LR_Predict)

accuracyLR = (LR_Result(1,1) + LR_Result(2,2)) / sum(sum(LR_Result))

precisionLR = LR_Result(1,1) / (LR_Result(1,1) + LR_Result(2,1))

```

```

recallLR = LR_Result(1,1) / (LR_Result(1,1) + LR_Result(1,2))

specificityLR = LR_Result(2,2) / (LR_Result(2,2) + LR_Result(2,1))

f1scoreLR = 2 * precisionLR * recallLR / (precisionLR + recallLR)

%% Confusion matrix LR

M = confusionchart(LR_Result)
title('LR in Testing Confusion Matrix');

%% Could manually using lassoglm be better ? Experementing <https://www.mathworks.
com/help/stats/lassoglm.html https://www.mathworks.com/help/stats/lassoglm.html>

rng('default') %
[B,FitInfo] = lassoglm(XTrain1BS,YTrain1BS,'binomial',...
    'NumLambda',25, 'CV',10);

lassoPlot(B,FitInfo,'PlotType','CV');
legend('show','Location','best')

% The plot identifies the minimum-deviance point with a green circle and dashed line
as a function of the
% regularization parameter Lambda. The blue circled point has minimum deviance plus no
more than one standard deviation.
%%
lassoPlot(B,FitInfo,'PlotType','Lambda','XScale','log');
% shhow nonzero model coefficients as a function of the regularization parameter
Lambda.
%%
indx = FitInfo.Index1SE;
B0 = B(:,indx);
nonzeros = sum(B0 ~= 0);
% Find the number of nonzero model coefficients at the Lambda value with minimum
deviance plus one standard deviation poin
%%
cnst = FitInfo.Intercept(indx);
B1 = [cnst;B0];
%%
figure(8)
preds = glmval(B1,trainDataMatrix,'logit');
histogram(trainTagMatrix - preds)
title('Residuals from lassoglm model')
% plot residuals from lassoglm model
%%
predictors = find(B0)
% indices of nonzero predictors
%% Experementing

[B,FitInfo] = lassoglm(XTrain1BS,YTrain1BS,'binomial',...
    'NumLambda',25, 'CV',10);
lassoPlot(B,FitInfo,'PlotType','CV');
legend('show','Location','best')
rng(1);
cvL2 = crossval(L2);
YpredL2=kfoldPredict(cvL2);

confMatL2 = confusionmat(YTrain1BS,YpredL2);

accuracyL2 = (confMatL2(1,1) + confMatL2(2,2)) / sum(sum(confMatL2))

```